

Software Requirements Specification (SRS)

PhishGuard: An Explainable Browser Extension for Real-Time Phishing Detection

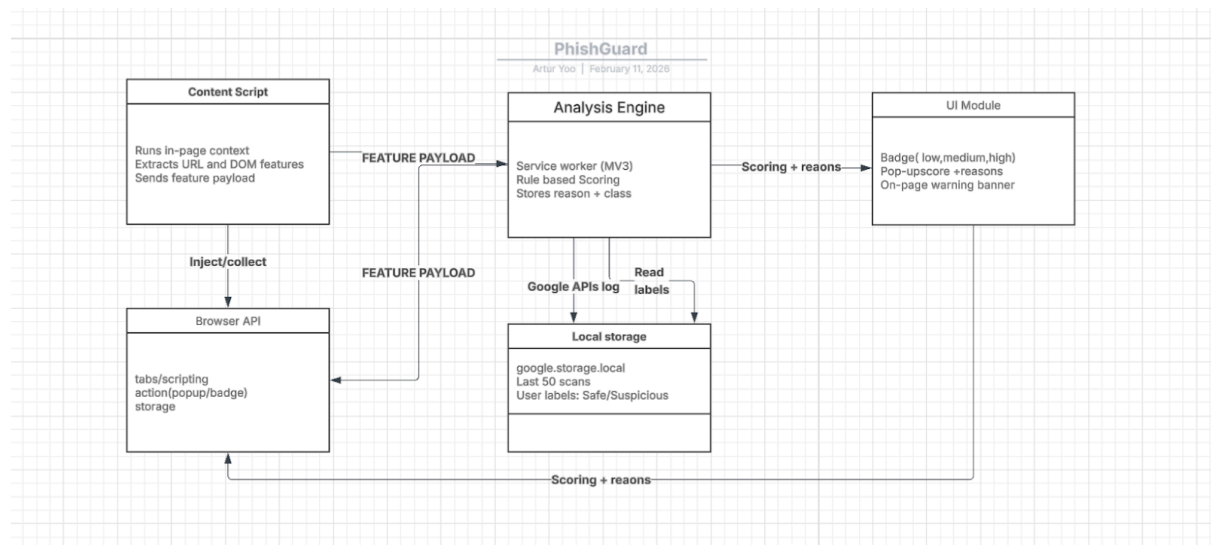
Author: Artur

Course: CMSI 4072 — Senior Project II

Date: 11 Feb 2026

5.1 Requirements Introduction

PhishGuard is a Google Chrome browser extension that analyzes webpages in real time to detect phishing indicators and present a risk classification to the user before credentials are submitted. The system extracts URL features and page-level structural features, computes a deterministic phishing risk score, and communicates risk classification through browser-based user interface components.



System Workflow Description

The system operates as a pipeline consisting of the following stages:

1. URL Extraction: The active tab URL is retrieved using the browser API.
2. Feature Extraction:
 - a. URL-based features are computed (hostname, subdomains, keywords).
 - b. DOM-based features are extracted (forms, password fields, title).
3. Risk Scoring: Extracted features are passed to the scoring engine, which computes a weighted risk score.
4. Classification: The score is mapped to a risk category (Low, Medium, High).
5. UI Feedback: The result is displayed via badge, popup interface, and on-page warning banner.

Module Responsibilities and Data Flow

- Content Script: Extracts DOM and URL features → outputs feature set
- Analysis Engine: Computes risk score → outputs score + indicators
- Storage Module: Stores scan results and user labels
- UI Module: Displays classification and explanations to the user

Data exchanged between modules consists of structured feature objects, numeric risk scores, and indicator lists.

5.2 Functional Requirements

5.2.1 URL Feature Extraction

Description:

The system shall analyze the structure and composition of the active webpage URL.

Requirements:

- 5.2.1.1 The system shall extract the full URL of the active tab.
 - 5.2.1.2 The system shall parse and store the hostname of the URL.
 - 5.2.1.3 The system shall determine whether the hostname is an IPv4 address.
 - 5.2.1.4 The system shall compute the number of subdomains in the hostname.
 - 5.2.1.5 The system shall detect the presence of punycode encoding in the hostname.
 - 5.2.1.6 The system shall compare the URL string against a predefined list of at least 20 phishing-related keywords.
-

5.2.2 Page Structure Analysis

Description:

The system shall analyze DOM-level features associated with phishing behavior.

Requirements:

- 5.2.2.1 The system shall detect the presence of at least one `<input type="password">` field.
 - 5.2.2.2 The system shall detect all HTML `<form>` elements on the page.
 - 5.2.2.3 The system shall determine whether any form submission target domain differs from the page's hostname.
 - 5.2.2.4 The system shall extract the page title string for keyword comparison.
-

5.2.3 Risk Scoring Engine

Description:

The system shall compute a phishing risk score using deterministic rule-based weighting.

Requirements:

- 5.2.3.1 The system shall assign a numeric weight to each detected phishing indicator.
 - 5.2.3.2 The system shall compute a total risk score between 0 and 100.
 - 5.2.3.3 The system shall classify risk as:
 - Low: 0–29
 - Medium: 30–59
 - High: 60–100
 - 5.2.3.4 The system shall store the contributing indicators for each computed score.
-

5.2.4 Warning and Notification Interface

Description:

The system shall communicate risk results clearly to the user.

Requirements:

5.2.4.1 The system shall display an on-page warning banner when risk classification is High.

5.2.4.2 The system shall display a colored extension badge indicating current risk classification.

5.2.4.3 The system shall provide a popup interface displaying:

- Numeric risk score
- Risk classification
- List of contributing indicators

5.2.4.4 The system shall allow the user to dismiss warning banners.

5.2.5 Scan Logging and Feedback

Description:

The system shall support local logging of scan results.

Requirements:

5.2.5.1 The system shall store the 50 most recent scan results in local browser storage.

5.2.5.2 The system shall allow the user to label a scanned page as “Safe” or “Suspicious.”

5.2.5.3 The system shall display stored scan results in the popup interface.

5.2.6 Privacy Constraints

Description:

The system shall operate without collecting sensitive user data.

Requirements:

5.2.6.1 The system shall perform all feature analysis locally within the browser.

5.2.6.2 The system shall not capture or store user-entered credentials.

5.2.6.3 The system shall not transmit browsing data to external servers.

5.2.7 Error Handling and Fallback Behavior

5.2.7 Error Handling and Fallback Behavior

5.2.7.1 The system shall handle failure to access webpage content without crashing.

5.2.7.2 The system shall display a neutral or unknown risk state when analysis cannot be completed.

5.2.7.3 The system shall handle missing or malformed URLs gracefully.

5.2.8 Accessibility Requirements

5.2.8.1 The system shall use color schemes distinguishable by color-blind users.

5.2.8.2 The system shall include text labels in addition to color indicators.

5.2.8.3 The system shall ensure UI elements are readable at standard browser zoom levels.

5.2.9 Rule and Keyword Management

5.2.9.1 The system shall allow updating of phishing keyword lists.

5.2.9.2 The system shall store keyword lists locally.

5.2.9.3 The system shall apply updated rules without requiring extension reinstallation.

5.3 Performance Requirements

Performance requirements define operational constraints on system behavior.

5.3.1 Page Analysis Latency

5.3.1.1 The system shall complete feature extraction and risk computation within 2000 milliseconds of page load completion.

5.3.2 Resource Utilization

5.3.2.1 The system shall not consume more than 100MB of RAM during operation.

5.3.2.2 The system shall not increase page load time by more than 5% under standard browsing conditions.

5.3.3 Storage Limits

5.3.3.1 The system shall limit stored scan records to a maximum of 50 entries.

5.3.5 CPU Utilization

5.3.5.1 The system shall not exceed 10% CPU usage during active scanning under normal conditions.

5.3.6 Repeated Scan Stability

5.3.6.1 The system shall maintain consistent performance over repeated scans within a browsing session.

5.3.6.2 The system shall not degrade in performance after scanning at least 50 pages consecutively.

5.3.7 High-Complexity Pages

5.3.7.1 The system shall complete analysis within 3000 milliseconds for pages with large DOM structures.

5.4 Environment Requirements

5.4.1 Development Environment Requirements

- Operating System: macOS, Windows, or Linux
 - Browser: Google Chrome (latest stable version)
 - Programming Language: JavaScript or TypeScript
 - IDE: VS Code or equivalent
 - Version Control: Git
 - Testing Scripts: Python 3.x for dataset evaluation
-

5.4.2 Execution Environment Requirements

- Chrome or Chromium-based browser supporting Manifest V3
- Desktop or laptop computer
- Minimum 4GB RAM
- Internet connection for browsing